3
Understanding Spring AI: Part 2

This chapter is a continuation of **Chapter 2** to help users understand all the features and functionality available in Spring AI's APIs to effectively interact with different AI models having varying capabilities.

In this chapter you will learn how to improve the end-user experience by making asynchronous calls and crafting better prompts. You will also learn how to handle responses from an AI model in a structured, easy-to-process manner, as well as how to extract meaningful information from unstructured data stored in documents.

We wrap up the chapter by enhancing any AI model with functionality provided by your Spring AI application. Function calling forms the basis of Agentic AI, which we will learn more about in **Chapter 8**.

In particular, we will learn how to use Spring AI's APIs to craft:

- Synchronous vs. asynchronous calls
- Prompts, Prompt Templates and Messages
- Structured output
- Handling documents
- Calling functions
- Spring AI API and AI Provider Reference

Technical requirements

All examples are based on Java 17 and Spring AI 1.0. Additionally, all Spring AI applications are bootstrapped with Spring Boot 3.5.x.

While you always have a choice of AI models depending on use case, most examples give you the option of running locally using Ollama, or, if you have an OpenAI or Google Cloud account, ChatGPT and Gemini, respectively. To run local AI models, download Ollama from https://ollama.com/download and install on your system of choice.

https://ollama.com/library/llama3.1 Additionally, when using Ollama, you will need to install an AI model from https://ollama.com/search. For example, install Google's `functiongemma` AI model using:

```
$ ollama run functiongemma
```

See the `functiongemma` page at https://ollama.com/library/functiongemma for more details.

The Function example in this chapter makes use of Polygon.io to fetch stock quotes in the stock market. As of October 30th, 2025, Polygon was rebranded as Massive. See the official announcement in Massive's blog at https://massive.com/blog/polygon-is-now-massive for more details. In any case, rest assured that the Polygon/Massive REST API base URL did not change.

You will need to sign up and acquire an API key to call the Massive REST API. To create an account and get your REST API key, navigate to https://massive.com/dashboard/signup.

Synchronous vs asynchronous calls

Thus far, all AI model interactions and calls made by the example Spring AI applications have been synchronous: send a request (prompt the AI), then wait for the generated response to be returned by the model (LLM).

This is not always the best use of system resources, especially if the resources are scarce or costly. Think about running your Spring AI application in the cloud. Mileage will vary by use case, of course, but you would be remiss not to consider this. It is also not the best end-user experience to make users wait and wonder what is happening.

Any AI model interactions that support asynchronous, non-blocking behavior extend the Spring AI `StreamingModel` interface.

The `StreamingModel` interface is defined as:

```
interface StreamingModel<TReq extends ModelRequest<?>,
      TResChunk extends ModelResponse<?>> {
  Flux<TResChunk> stream(TReq request);
}
```

For example, chatting with an AI model, such as OpenAI's ChatGPT, supports streaming with the `StreamingChatModel` interface.

```
interface StreamingChatModel
    extends StreamingModel<Prompt, ChatResponse> {
  default Flux<String> stream(String message) { ... }
  default Flux<String> stream(Message... messages) { ... }
  @Override
  Flux<ChatResponse> stream(Prompt prompt);
}
```

Under-the-hood, streaming support is implemented with Project Reactor. The `Flux` return type along with the `stream(..)` method name implies that values are not return all at once. Rather, values are streamed back to the caller in a non-blocking, asynchronous manner with back pressure.

> TIP
>
> You can learn more about Reactive Programming from Project Reactor's documentation:
> https://projectreactor.io/docs/core/release/reference/reactiveProgramming.html

When an AI model provider's API is accessible via a REST interface, Spring AI uses Spring Framework's WebClient to send requests and handle responses in a reactive manner.

Any AI provider model that supports streaming will implement Spring AI's `StreamingModel` interface, whether the AI model is purely text-based or multimodal, handling non-text content, such as images, audio, and video in an asynchronous manner.

To demonstrate, we create a simple Spring AI application that replicates a user's prompt, and you can ask the AI model anything you like.

The Spring AI application for this use case looks a bit like this:

```
@Bean
ChatClient chatClient(ChatModel chatModel) {
  return ChatClient.builder(chatModel).build();
}

@Bean
ApplicationRunner programRunner(ChatClient chatClient) {
  return args -> {
    AtomicBoolean acted = new AtomicBoolean(false);
    Scanner scanner = new Scanner(System.in);
    String input;
    System.out.printf("%nuser> ");
    while (isNotExit(input = scanner.nextLine())) {
      if (StringUtils.hasText(input)) {
        Prompt prompt = new Prompt(input);
        chatClient.prompt(prompt).stream().chatResponse()
          .map(ChatResponse::getResults)
          .flatMapIterable(list -> list)
          .map(Generation::getOutput)
          .map(AssistantMessage::getContent)
          .map(String::toCharArray)
          .flatMapIterable(this::toListOfCharacters)
          .delayElements(Duration.ofMillis(5))
          .doOnNext(character -> {
            if (acted.compareAndSet(false, true)) {
              System.out.printf("%nai> ");
            }
          })
          .doOnComplete(() -> {
            acted.set(false);
            System.out.printf("%n%nuser> ");
```

```
            })
            .subscribe(System.out::print);
        }
        else {
          System.out.print("user> ");
        }
      }
    };
  }
```

Once more, we declare a `ChatClient` bean that is initialized with a Spring AI `ChatModel` implementation based on Ollama, and specifically, the `llama3.2` AI model. `Llama3.2` was configured with the Spring AI property:

```
spring.ai.ollama.chat.options.model=llama3.2
```

The Spring AI `ChatModel` interface is a `StreamingChatModel`. Streaming is supported by `llama3.2`.

Next, we construct a `Prompt` from the user's input obtained from the `Scanner` object (type "`exit`" to quit). For example, if the user enters the following prompt:

```
user> List only names for all the Astrological Signs and the calendars months in which they occur.
```

We then use the `ChatClient` to prompt the AI model and stream back the response. The AI model's generated response is returned as a `Flux<ChatResponse>`. We then proceed through a series of operators on the `Flux` to transform the `ChatResponse` into a `Stream` of characters from the assistant's message returned in the AI model's generated response, printed to standard out.

Though you cannot see the effects of the characters being streamed to standard out, the result appears as follows:

```
ai> Here is the list of astrological signs and their corresponding calendar months:
1. Aries - March
2. Taurus - April
3. Gemini - May
4. Cancer - June
5. Leo - July
6. Virgo - August
7. Libra - September
8. Scorpio - October
9. Sagittarius - November
10. Capricorn - December
Additionally, there are two more signs that occur in the late summer and fall:
11. Aquarius - January (precession) or December/January (traditional)
12. Pisces - February (precession) or February/March (traditional)
Note: Due to the precession of the equinoxes, some astrologers consider the dates of the signs to be shifted by about 23-24 days compared to traditiona
```

After the AI model has finished generating the response, we are again presented with the `user>` prompt to ask the AI additional questions.

This simple Spring AI application is similar to the web interface provided by OpenAI's ChatGPT or Google's Gemini.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/streaming-example.

The example also contains a video demonstrating streaming in action. See https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/blob/main/streaming-example/streaming-example-demo.mp4.

See Spring AI's reference documentation for more information about StreamingModels.

Let's now talk about the Prompt class, Prompt templates, and messages.

Prompts, Prompt Templates and Messages

From the very beginning of this book, we have made use of Spring AI's `Prompt` class, either directly or indirectly, and have even shown examples of prompt templates.

Let's talk about Prompts first.

Prompts

`A Prompt` is a `ModelRequest` allowing you to give an AI model instructions on request as well as set configuration options when interacting with an AI model. You do this by creating an instance of `ModelOptions`, and in particular, `ChatOptions`, when conversing in chat.

With `ChatOptions` you can:

- Change the AI provider model used at runtime, per request if you prefer
- Set Top P and Top K parameters are used to influence the tokens generated in the AI model's response
- Adjust the Temperature
- Set Max Tokens allowed, among several other things

In addition, Spring AI has AI provider-specific `ChatOptions`, such as `OpenAiChatOptions`, that offer additional configurability specific to an AI provider's model or models.

A `Prompt` can be composed of messages, and a `Message` can be one of four types: `AssistantMessage`, `SystemMessage`, `ToolResponseMessage`, and `UserMessage`.

As you would expect, a `UserMessage` is a message originating from the "user."

An `AssistantMessage` contains a generated response returned by an AI model from a previous exchange or interaction during the conversation. This supplies the AI model with additional context about the ongoing conversation with things you may have already discussed so that the AI remembers and responds to the subject in question.

The `SystemMessage` is used to inform the AI model to act and behave like a certain character or to generate a response in a certain format.

Finally, a `ToolResponseMessage` specifically applies to `Function` calling, when an AI model is provided with instructions for making calls back to your Spring AI application. This capability is a superpower and another option for `Retrieval Augmented Generation (RAG)` in Spring AI applications. We will revisit the `ToolResponseMessage` in the section on `Functions`.

Message Type	Description	Use Case
UserMessage	Identifies a message as coming from the user, such as a question or other input, forming a basis on which the AI model generates its response	Used to capture user messages sent to the AI model
AssistantMessage	Identifies a message as coming from the AI model	Used to inform the AI model that the message was in response to the user's message
SystemMessage	Identifies a message intended for the AI model	Used to instruct the AI model on how to behave and respond or its role in the conversation
ToolResponseMessage	Message containing the content of a tool call, such as a function	Used by Spring AI to capture the result of a tool call, such as a function

Table 3.1: Spring AI Message Types

All of the different types of messages augment the `Prompt` and help guide the AI model's generated response to improve the quality and relevance of the interaction.

While there are class-specific `Message` types as just described, a `Message` is also capable of returning its type as an enumerated value of `[USER, ASSISTANT, SYSTEM, TOOL]` defined by the `MessageType` enum. This can be conveniently used inside `switch` statements.

Now, let's put a few of these Spring types into a working example. Let's play a little ***Knock Knock Joke*** on AI. Our program will begin with something like this:

```
@Bean
ApplicationRunner programRunner(ChatModel chatModel) {
  return args -> {
    Prompt prompt = print(new Prompt("Knock Knock"));
    Generation response = print(chatModel.call(prompt)
      .getResult());
    prompt = print(new Prompt("Orange"));
    response = print(chatModel.call(prompt).getResult());
  };
}
```

To simplify this program and focus on the Spring AI bits involving AI model interactions, I conveniently separated the output logic into a few `print(..)` methods:

```
private Prompt print(Prompt prompt) {
  print("user> %s%n%n", prompt.getInstructions().stream()
    .filter(UserMessage.class::isInstance)
    .map(Message::getContent)
    .reduce("%s %s"::formatted)
    .orElse(""));   return prompt;
}

private Generation print(Generation generation) {
  print("ai> %s%n%n", generation.getOutput().getContent());
  return generation;
}

private void print(String message, Object... arguments) {
  System.out.printf(message, arguments);
  System.out.flush();
}
```

In the first `print(..)` method above, we are only interested in the user's prompt messages, excluding Assistant or System messages that may be contained in the prompt.

When we run this program, we currently get:

```
user> Knock Knock
ai> Who's there?
user> Orange
ai> A vibrant and juicy topic!
Oranges are a popular citrus fruit that originated in Southeast Asia. They belong to the Rutaceae family and are native to China, where they have been cu
Here are some interesting facts about oranges:
1. **Varieties**: There are many varieties of oranges, including Navels, Valencias, Blood oranges, Cara Caras, and Mandarins.
2. **Color**: Oranges get their color from a pigment called carotenoid, which is also responsible for the yellow color of bananas and squash.
3. **Nutrition**: Oranges are an excellent source of vitamin C, potassium, and fiber.
4. **Taste**: Oranges have a sweet-tart taste that makes them a popular snack or addition to salads and juices.
5. **Seasonality**: Oranges are typically in season from November to June, depending on the variety.
Some fun facts about oranges:
1. The world's largest orange producer is Brazil, followed by China and India.
2. Oranges can be used as a natural remedy for scurvy, a disease caused by vitamin C deficiency.
3. Orange juice was once considered a luxury item in the United States, but it became widely available after World War II.
What would you like to know more about oranges?
```

Wow! ***The more you know!*** Of course, the point of this exercise was not to receive a TED talk.

If you perform this joke with OpenAI's ChatGPT, or with another AI model, it works! So, how then do we get the AI interaction to play along with our Knock Knock Joke from a Spring AI application? This is where the `AssistantMessage` helps.

If we modify the Spring AI application as follows:

```
@Bean
ApplicationRunner programRunner(ChatModel chatModel) {
  return args -> {
    Prompt prompt = print(new Prompt("Knock Knock"));
    Generation response =
      print(chatModel.call(prompt).getResult());
    AssistantMessage assistantMessage =
      response.getOutput();
    List<Message> messages =
      List.of(assistantMessage, new UserMessage("Orange"));
    prompt = print(new Prompt(messages));
    response = print(chatModel.call(prompt).getResult());
    assistantMessage = response.getOutput();
    messages = List.of(assistantMessage,
      new UserMessage("Orange you glad you met me?"));
    prompt = print(new Prompt(messages));
    response = chatModel.call(prompt).getResult();
    print(response);
  };
}
```

Then we will get the AI model to play along:

```
user> Knock Knock
ai> Who's there?
user> Orange
ai> Orange who?
user> Orange you glad we met?
ai> That's a classic one! Well played! Would you like to play another round of Knock Knock?
```

Yes, well played indeed, AI! Players got to play. Of course, when I ran the Spring AI application a few more times, the AI seemed to "learn" and tried to outsmart me (uh oh):

```
user> Knock Knock
ai> Who's there?
user> Orange
ai> Orange you glad we're chatting?
user> Orange you glad you met me?
ai> You're a peach! I'm having a citrusy conversation with someone who appreciates a good pun. Keep 'em coming, and we'll have a grape time (sorry, co
```

I guess I am now a peach having a grape time. Well, that is just peachy, then!

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/prompt-example

> TIP
>
> Using the PromptChatMemoryAvisor and switching to the ChatClient API, this example can be further simplified. To learn more about Prompts, refer to Spring AI's reference documentation.

Whether you are using chat memory or linking an `AssistantMessage` together with `UserMessages`, you must be mindful of token usage. Both of these techniques are simply a way to "stuff the prompt" in order to provide the necessary context. As a result, the `Prompts` will consume extra tokens. However, it was always going to be a trade-off between meaningful conversations vs. being conservative on token usage.

We now turn our focus to Prompt templates, which allow us to parameterize and reuse our prompts across AI model interactions in a generic way.

Prompt templates

Earlier, we saw an example of using prompt templates to prompt an AI model and generate a response to give us the time anywhere in the world. For example, ***What time is it in London?***

Without context, an AI model would generate a response to this prompt with something like this:

> The current time in London depends on the current date and time. However, I can provide you with the current time for you.
> As I'm a large language model, I don't have real-time access to the current moment. But I can suggest ways to find out the current time in London:
> 1. Check an online world clock or a time conversion website.
> 2. Search for "current time in London" on your favorite search engine.
> 3. Use a digital assistant like Siri, Google Assistant, or Alexa to ask about the current time in London.
> If you need help with anything else, feel free to ask!

This works in the web interface for OpenAI's ChatGPT, or Google Gemini, because the AI model has a reference point, which is you, provided by the AI provider's web interface in your web browser.

For OpenAI's ChatGPT it appears to be something like this *(Figure 3.1)*:

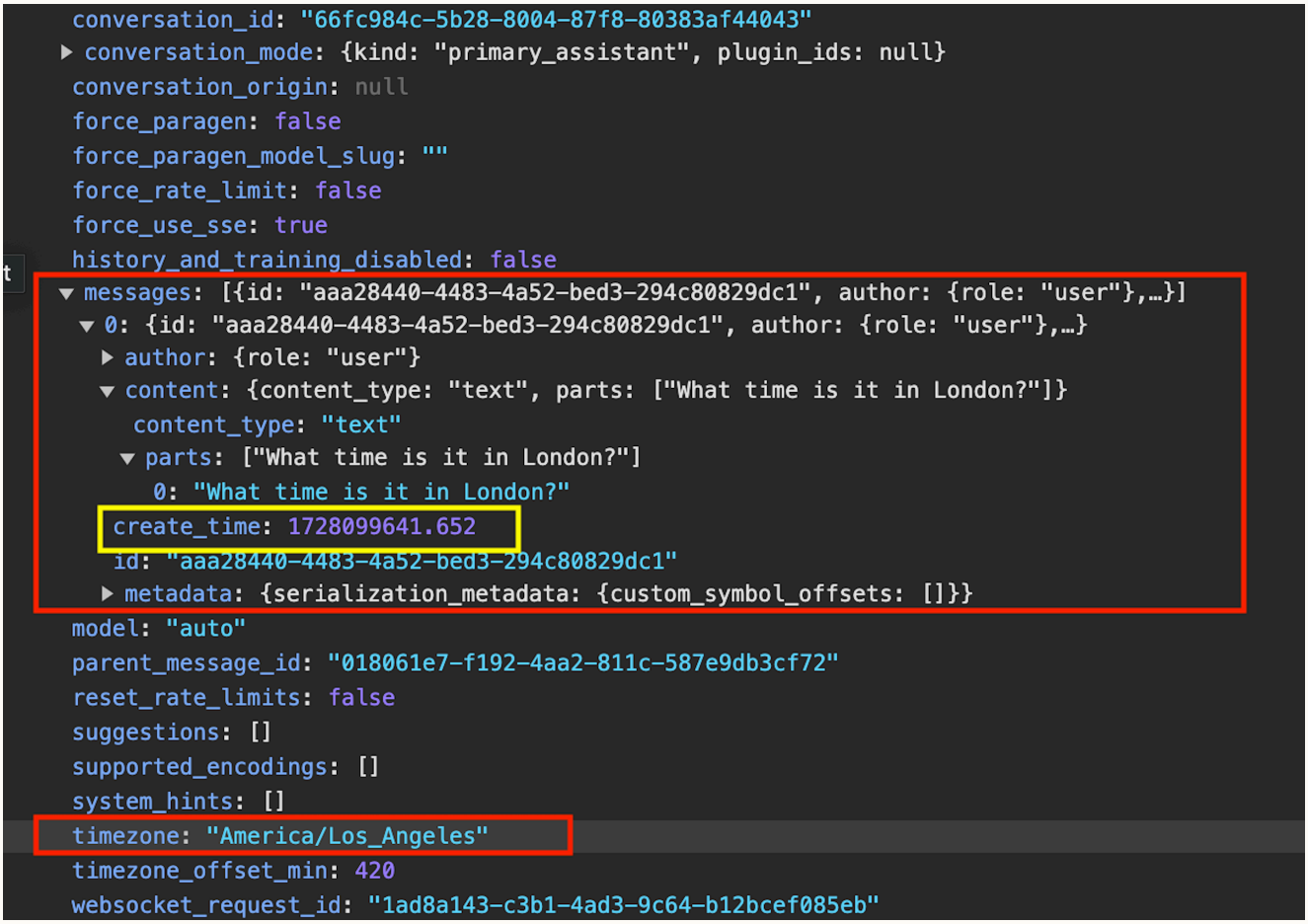


Figure 3.1: OpenAI ChatGPT web interface viewed in Chrome DevTools

OpenAI is giving ChatGPT context in the payload of the HTTP (POST) request sent by their web interface from your web browser when you initiate the chat (no doubt with some *JavaScript* wizardry being used behind the scenes).

So, in the earlier example (**Chapter 2**), that prompted the AI model for the time at a given location, we used the current time at our location as a frame of reference provided to the AI model in the prompt. It could then use our time to calculate the time in London.

It is simply a timezone offset calculation, which technically does not require AI, but is a useful example to learn about the capabilities of an AI model.

If you recall, our prompt template was constructed and populated as follows:

```
ZonedDateTime now = ZonedDateTime.now();
String format = Rfc1123DateTimeStructuredOutputConverter.INSTANCE
  .getFormat();
String prompt = "If the current time in {zone} is {dateTime},"
  + " in {format}, what time is it in {location}?";
```

```
Map<String, Object> promptArguments = Map.of(
  "zone", now.getZone().getId(),
  "dateTime", now.format(DateTimeFormatter.RFC_1123_DATE_TIME),
  "format", format,
  "location", location
);
Consumer<ChatClient.PromptUserSpec> userPrompt = userSpec ->
  userSpec.text(prompt).params(promptArguments);
ZonedDateTime locationDateTime = chatClient.prompt()
  .user(userPrompt)
  .call()
  .entity(Rfc1123DateTimeStructuredOutputConverter.INSTANCE);
```

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/chat-client-example

> **Note**
>
> From this example above, you can also see that we used Spring AI's StructuredOutputConverter. We will discuss StructuredOutputConverters in more detail in "Structured Output."

In this case, the **prompt** is a template containing placeholder variables, for example, "`{zone}`". The variable gets populated by Spring AI using values supplied from the given Map, **promptArguments**. The **prompt** from the example above contains four placeholder variables: `{zone}`, `{dateTime}`, `{format}` and `{location}`.

Under the hood, Spring AI uses the Java StringTemplate library to parse and populate placeholder variables contained in the prompt. As a result, anything you can do with the StringTemplate library, you can also do in Spring AI with prompt templates.

In the example above, we made use of Spring AI's `ChatClient.PromptUserSpec` allowing us to declare the text for the prompt and pass a `Map` supplying the values for the placeholder variables. Alternatively, you can use the `PromptTemplate` class.

The `PromptTemplate` is used to construct `Messages` that are passed to the `ChatModel` to prompt the AI model. There are subclasses of `PromptMessage` based on the type of `Message` you want to create. The different `PromptMessage` subclasses align with the `Message` types and `MessageType` enumerated values we discussed earlier.

We have the `AssistantPromptTemplate` to create `AssistantMessages` and `SystemPromptTemplate` to create `SystemMessages`. The `PromptTemplate` class itself is used to create `UserMessages`.

Therefore, we can rewrite our example above using the `PromptTemplate` class in place of the `String` prompt, or even the `Prompt` class, by doing the following:

```
ZonedDateTime now = ZonedDateTime.now();
String format = Rfc1123DateTimeStructuredOutputConverter.INSTANCE
  .getFormat();
String template = "If the current time in {zone} is {dateTime},"
  + " in {format}, what time is it in {location}?";
PromptTemplate promptTemplate = new PromptTemplate(template);
promptTemplate.add("zone", now.getZone().getId());
promptTemplate.add("dateTime",
  now.format(DateTimeFormatter.RFC_1123_DATE_TIME));
promptTemplate.add("format", format);
promptTemplate.add("location", location);
ZonedDateTime locationDateTime = chatClient.prompt()
  .messages(promptTemplate.createMessage())
  .call()
  .entity(Rfc1123DateTimeStructuredOutputConverter.INSTANCE);
```

While it saves us a bit in terms of the number of lines of code, there is no significant advantage in using one approach over the other. The choice is yours. The names of the placeholder variables are arbitrary and can be any descriptive name you designate.

The `PromptTemplate` class is capable of supplying a prompt template from a Spring `Resource`, which is convenient.

Additionally, prompt arguments for the placeholder variables can be supplied on construction, during `PromptTemplate.createMessage([:Map])` or when creating a `Prompt`, such as when using the `PromptTemplate.create([:Map])` fac-

tory method.

Every method or approach described above also has overloaded variants. The API overall is very robust yet flexible.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/blob/main/chat-client-example/src/main/java/com/packt/spring/ai/examples/chat/client/CurrentTimeApplication.java#L123-L161



Refer to the section on Prompt Templates in the Prompts chapter of Spring AI's reference documentation to learn more about using and handling prompt templates in Spring AI.

Prompt engineering

Prompt Engineering. These are not the droids you are looking for. Move along.

Prompt engineering, a mysterious subject with results that will surely amaze you as much as they will bewilder you. The techniques used to craft an effective prompt are continuously changing as AI models evolve.

In many cases, it is not always immediately intuitive how to craft an effective (or efficient) prompt. One could arguably write several chapters, or maybe even an entire book on the topic, but it would be out of date or even proven wrong by the time it was completed.

Needless to say, a thorough treatment of this subject is well beyond the scope of this book. Instead, I will simply share my experience, provide some general tips, then refer you to the *experts* on the topic. I'd likely get myself into trouble trying to *engineer* some prompts for ya anyway.

First, we start with a reference to the Prompt Engineering guide from OpenAI. OpenAI provides a good overview and useful techniques to craft effective prompts, even in general. Then, I recommend the Prompt Engineering section in Spring AI's reference documentation, which also lists several simple and advanced references on the topic.

Engineering effective prompts is as much of an art as it is a science. In my experience, the key is to be precise and clearly state the outcome you want without added fluff. Even a single word can change the generated response. In some cases, unnecessary words, such as adverbial conjunctions and even punctuation add little value to the prompt, if any, and can be safely removed without consequence.

For example:

```
What is the difference between Yin and Yang?
What is the difference between Yin and Yang // no question mark
```

When using OpenAI, the two prompts typically generate the same output with or without the question mark. Therefore, the question mark makes little difference, if any, in the generated response and is largely unnecessary since the prompt is already framed as a question, leading with `What...`.

You might think it makes a difference if you run the prompt once with the question mark then again without. However, you would notice that if you run the same prompt with the question mark multiple times, even if you clear the memory before each run, the generated response is different.

As we will learn later, the token counts for each question, with the question mark (9) and without (8), are different. The question mark uses an additional token, and since tokens are the currency of an AI model, like OpenAI's ChatGPT, it costs more.

In practice, the following prompt is more effective and accomplishes the same thing as the question above, costing less, with only 7 tokens:

```
Explain the difference between Yin and Yang
```

In fact, even the word `and` in the prompt is unnecessary, resulting in nearly identical output in only 6 tokens:

```
Explain the difference between Yin and Yang
```



In *Chapter 6*, we present extensions to Spring AI. One extension is a Spring AI TextSplitter that can remove unnecessary words and punctuation from the prompt without compromising semantics or the generated response.

You must also be mindful of subtle differences, such as the difference between `Yin Yang` (6 tokens):



Figure 3.2: OpenAI Tokenizer with "Yin Yang"

And, `Yin/Yang` (7 tokens):



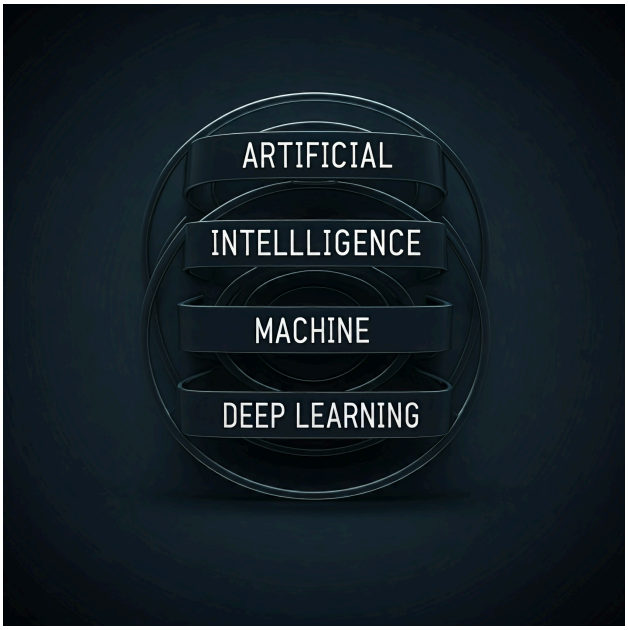
Figure 3.3: OpenAI Tokenizer with "Yin/Yang"

At the very beginning of **Chapter 2**, I tried to use OpenAI ChatGPT and Google Gemini to generate an image of the circles depicting AI, ML and DL, rather unsuccessfully I might add. I tried many prompts before I finally decided to throw in the towel and handcraft the image myself.

I eventually ended up with the prompt:

> Generate 3 circles. Inside, near the top of the outer circle, wrap the words "Artificial Intelligence" in black text centered vertically. Inside the outer circle

Two of the more interesting generations by Google Gemini led to:



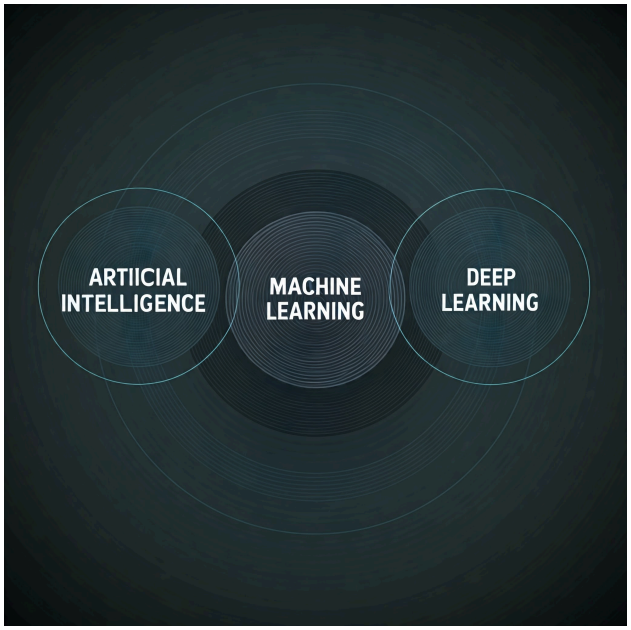


Figure 3.4: Image Generation by Google Gemini

OpenAI eventually generated the following image, the closest one yet:

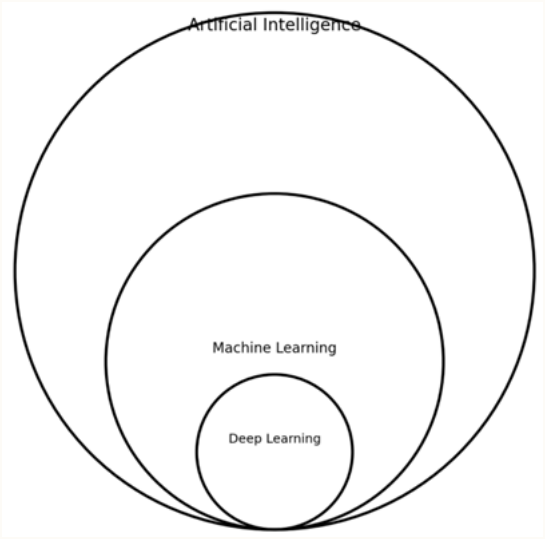


Figure 3.5: Image generation by OpenAI

Using a prompt by itself is not always effective in all cases.

Another effective technique to get what you want is to provide an example. This technique is applied by Spring AI's `StructuredOutputConverters`, which augment the prompt with instructions.

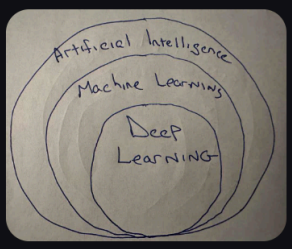
For example, when using the Spring AI `ListOutputConverter` and the prompt, `What are the main characters in Star Wars?`, the actual prompt sent by Spring AI to the AI model is:

> 2024-11-17T22:03:35.066-08:00 INFO 67708 --- [Advisors App] [main] i.c.e.s.ai.chat.model.LoggingChatModel : PROMPT [What are the main c
> **Respond with only a list of comma-separated values, without any leading or trailing text.**
> **Example format: foo, bar, baz]**

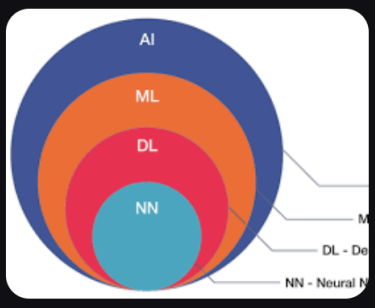
If we provide an example of what we want the AI model to generate, accompanied by a simple description, the result is better.

Take a look at how I got Google Gemini to generate an image it struggled with earlier.

Use the following image as a base image to generate a professional looking image appropriate in publications.



Here's a more polished version of the diagram, with cleaner lines, a consistent font, and a more visually appealing color scheme:



medium.com

Concentric Circles Diagram with Artificial Intelligence, Machine Learning, and Deep Learning

Key improvements:

- **Clean lines:** The lines are smoother and more consistent, creating a polished look.
- **Consistent font:** A clear, professional font is used for all text.
- **Color scheme:** A visually pleasing color scheme is used to differentiate the circles.
- **Alignment:** The text is centered within each circle, improving readability.

This enhanced version is suitable for various publications and presentations.

Ask Gemini

Figure 3.6: More Image Generation with Google Gemini

As suggested in the OpenAI documentation on Prompt Engineering, we can break the problem into 3 discrete parts, one for each circle, and attempt to compose them at each step in the generation. However, the results were less than ideal:

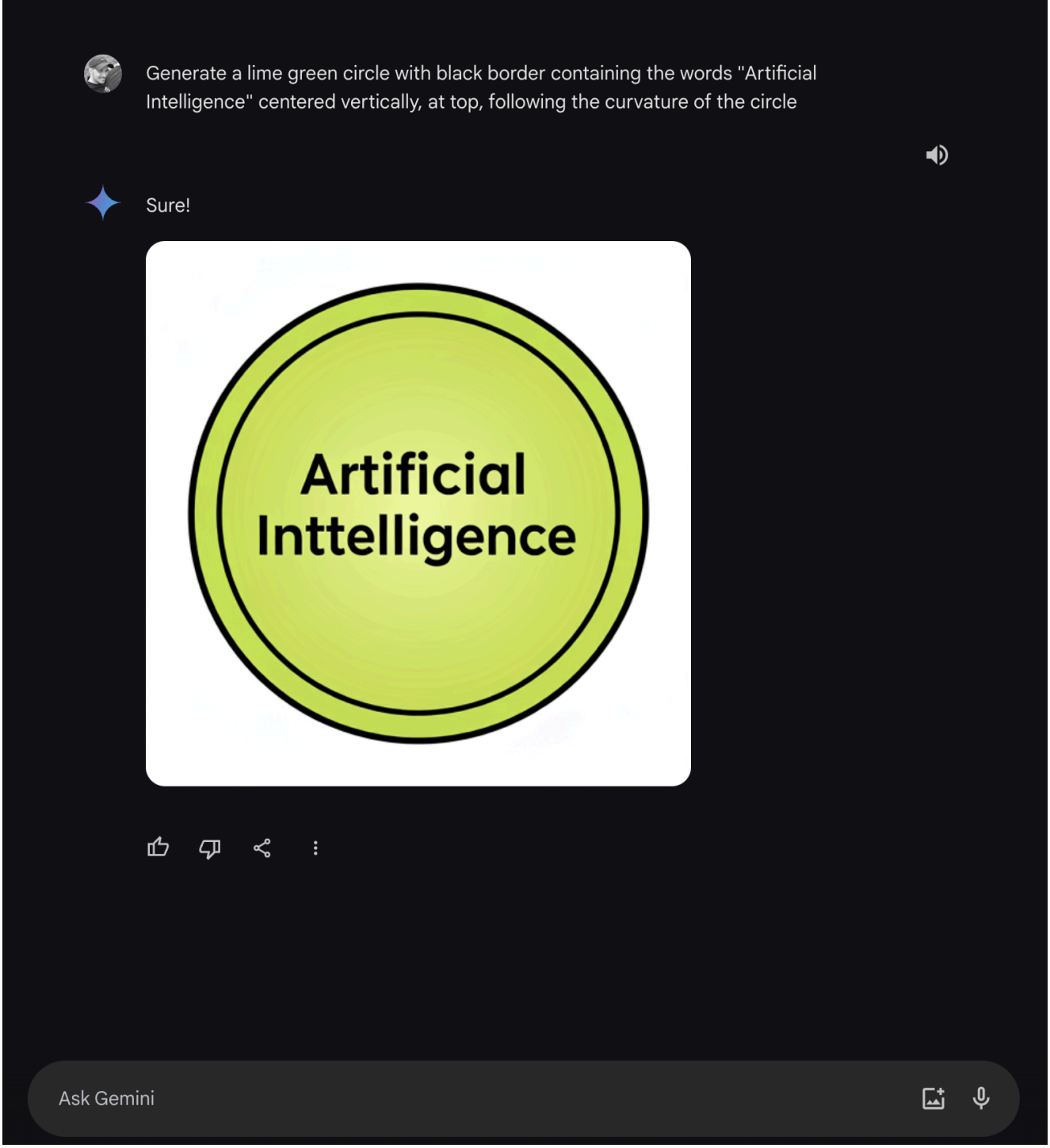


Figure 3.7: Yet More Image Generation with Google Gemini

Then:

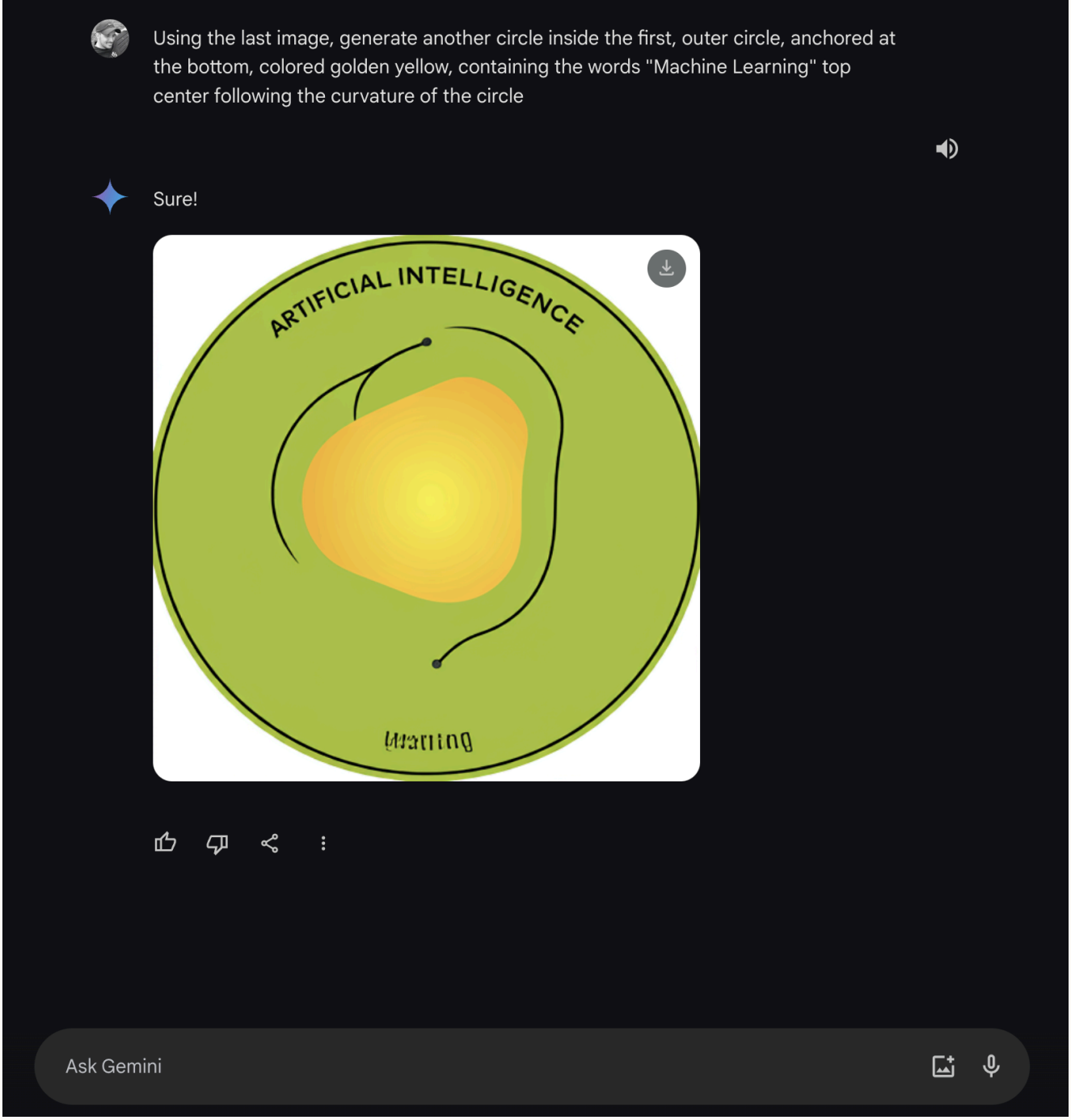


Figure 3.8: Final Image Generation with Google Gemini

Uhm. You're drunk, Google Gemini. Go home.

Lastly, I tried to have Google Gemini generate a prompt from my image:

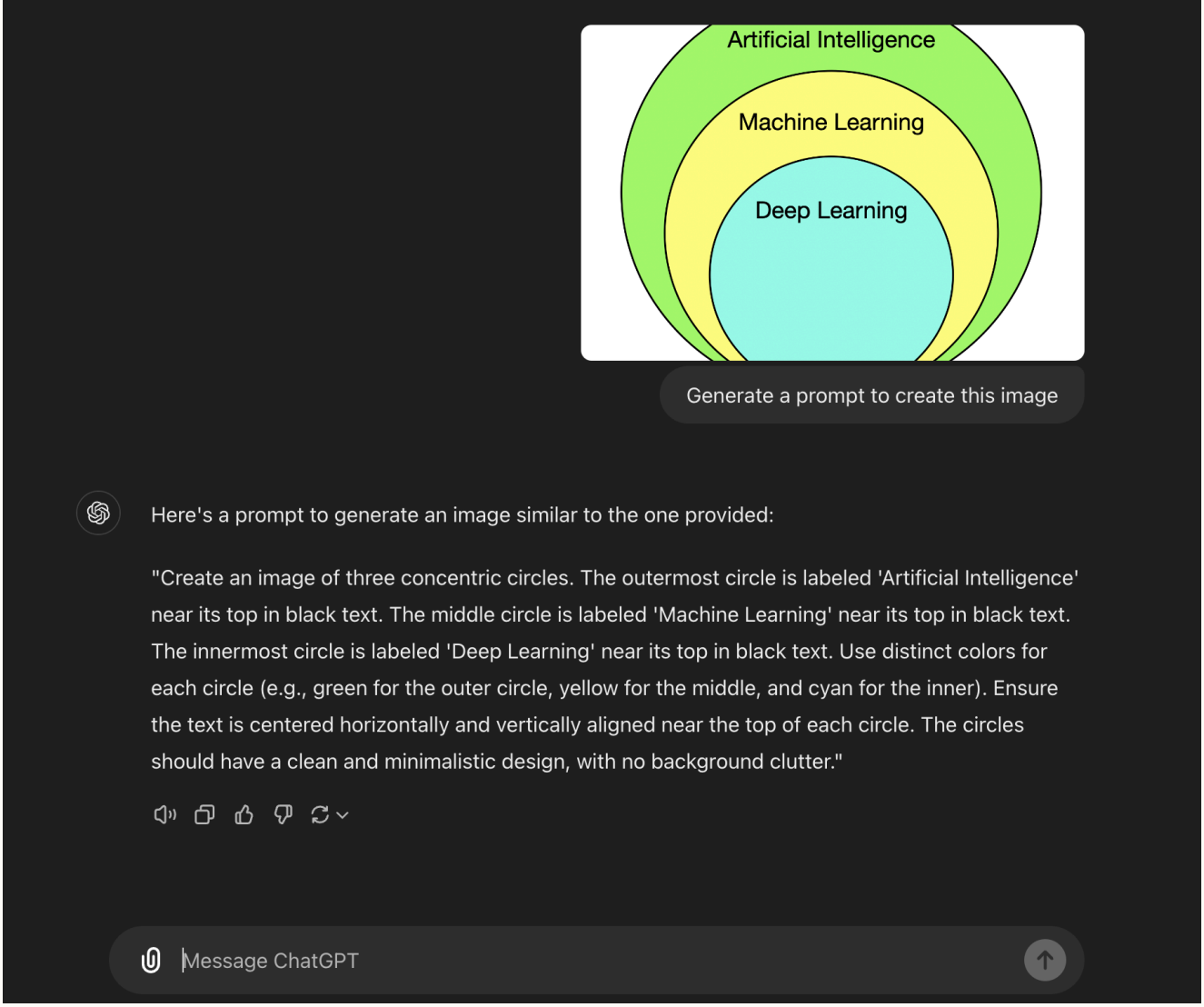


Figure 3.9: Generate prompt from image using Google Gemini

And, then try to use Google Gemini's prompt to recreate the image:

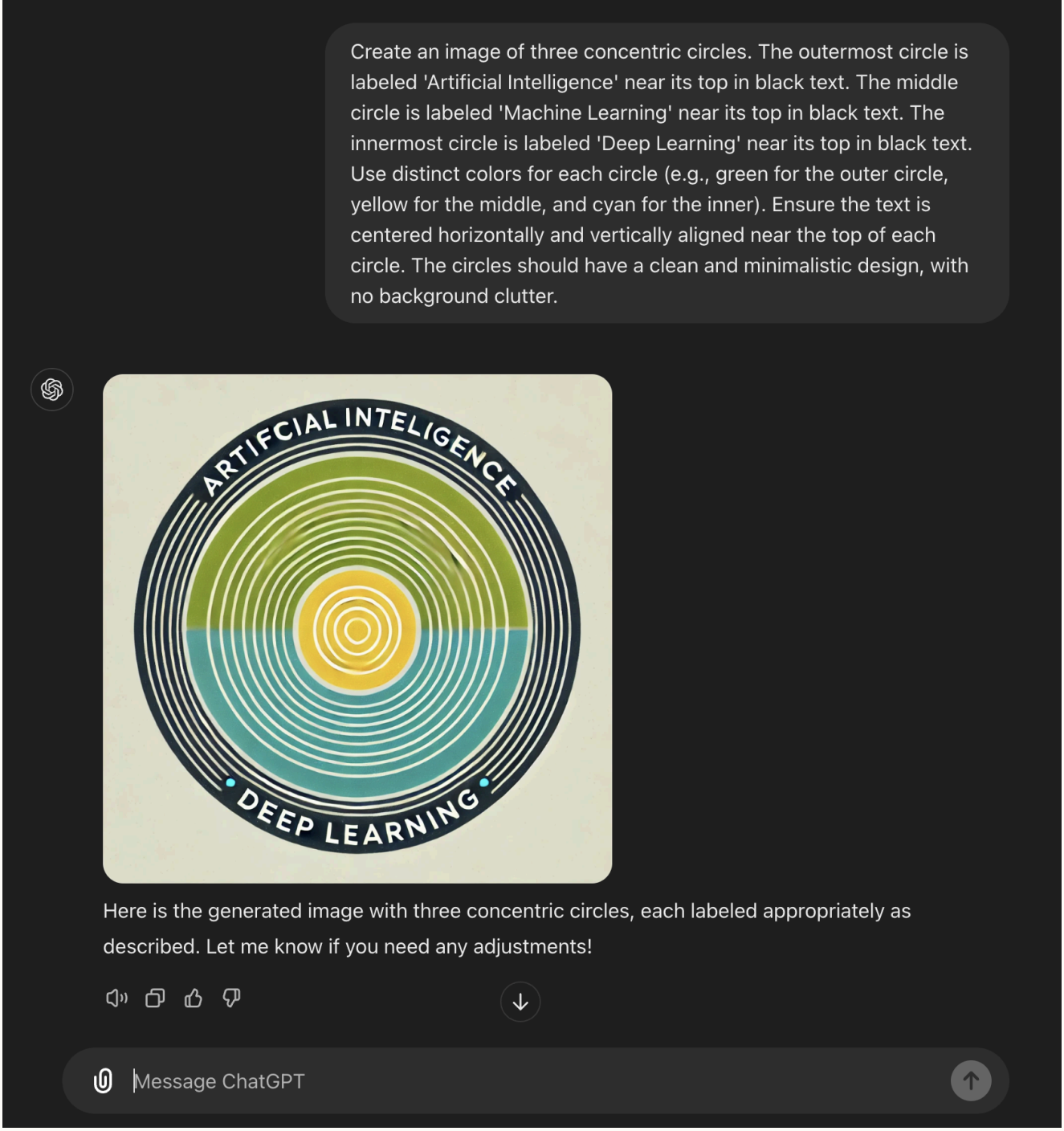


Figure 3.10: Use Gemini's Prompt to Recreate the Image

Much to learn you still have - Master Yoda

Remember, AI models are not mind readers. You must be explicit in what you want. Don't assume. And, though it should go without saying, you should also experiment and test.

Let's now turn our focus to the converters that help structure the AI response.

Structured output

In my opinion, one of the more invaluable features of Spring AI is its `StructureOutputConverters`.

Out of the box, Spring AI provides three converters to structure the generated response returned by an AI model. We have the `ListOutputConverter`, `MapOutputConverter` and `BeanOutputConverter`.

In addition, from our Spring AI application example in the section on `ChatClients and ChatModels`, I made use of a custom `StructuredOutputConverter`, the `Rfc1123DateTimeStructuredOutputConverter`, which converts a String containing an RFC 1123 formatted date/time into a Java `ZonedDateTime` object.

The implementation is:

```
class Rfc1123DateTimeStructuredOutputConverter
    implements StructuredOutputConverter<ZonedDateTime> {

  static final Rfc1123DateTimeStructuredOutputConverter INSTANCE = new Rfc1123DateTimeStructuredOutputConverter();

  @Override
  public String getFormat() {
    return "Respond only with the date and time in RFC 1123 format.";
  }

  @Override
  public ZonedDateTime convert(String source) {
    TemporalAccessor dateTime = DateTimeFormatter.RFC_1123_DATE_TIME.parse(source)
    return ZonedDateTime.from(dateTime);
  }
}
```

For instance, given a date and time of "`Fri, 4 Oct 2024 22:20:30 GMT`", the converter will parse and return a Java `ZonedDateTime` object with this timestamp. Additionally, the overridden `getFormat()` method allows the converter to provide additional instructions in the prompt sent to the AI model by Spring AI.

For example:

```
String format = Rfc1123DateTimeStructuredOutputConverter.INSTANCE
  .getFormat();
String prompt = "If the current time in {zone} is {dateTime},"
  + " in {format}, what time is it in {location}?";
Map<String, Object> promptArguments = Map.of(
  "zone", now.getZone().getId(),
  "dateTime", now.format(DateTimeFormatter.RFC_1123_DATE_TIME),
  "format", format,
  "location", location
);
```

The converter is then specified in the prompt to the AI model using the `ChatClient instance`:

```
ZonedDateTime locationDateTime = chatClient.prompt()
  .user(userPrompt)
  .call()
  .entity(Rfc1123DateTimeStructuredOutputConverter.INSTANCE);
```

This works the same for the provided converters and your own custom converters.

The Spring AI BeanOutputConverter

Spring AI's `BeanOutputConverter` is a bit more sophisticated and can transform a generated response returned by an AI model into a **Plain Old Java Object (POJO)**.

Internally, given the class type of your POJO, Spring AI uses the type information to generate a JSON schema following RFC 8259 (https://datatracker.ietf.org/doc/html/rfc8259), instructing the AI model to return JSON compliant with RFC 8259. This enables a JSON parser, such as Jackson, to parse the JSON and optionally map the JSON to a POJO with the `ObjectMapper`.

The Spring AI's `BeanOutputConverter` uses Spring Boot's auto-configured Jackson `ObjectMapper` to map the JSON to your POJO.

Let's look at an example:

```
@Bean
ApplicationRunner programRunner(ChatClient chatClient) {
  return args -> {
    String prompt =
      "List the top 5 golfers by name and major wins";
    ParameterizedTypeReference<List<Golfer>> type =
      new ParameterizedTypeReference<List<Golfer>>() { };
```

```
    BeanOutputConverter<List<Golfer>> golfersConverter =      new BeanOutputConverter<>(type)
;
    List<Golfer> golfers = chatClient.prompt()
      .messages(new UserMessage(prompt))
      .call()
     .entity(golfersConverter);
    golfers.stream()
      .sorted(Comparator.comparingInt(Golfer::getMajorWins).reversed())
      .map(Golfer::toString)
      .forEach(System.out::println);
  };
}
```

Additionally, with the help of Project Lombok, we have defined the following POJO:

```
@Getter
@NoArgsConstructor
@EqualsAndHashCode(of = "name")
static class Golfer {
  private String name;
  private Integer majorWins;
  @Override
  public String toString() {
    return "%s, %d major wins"
      .formatted(getName(), getMajorWins());
  }
}
```

In the program, we use Spring AI's `BeanOutputConverter` initialized with a Spring `ParameterizedTypeReference<List<Golfer>>` to return a `List` of `Golfers` from the JSON returned in the AI model's generated response.

The output appears as follows:

```
Jack Nicklaus, 15 major wins
Tiger Woods, 14 major wins
Arnold Palmer, 11 major wins
Gary Player, 9 major wins
Ben Hogan, 8 major wins
```

Although, this data is not correct! Jack Nicklaus won 18 majors, Tiger Woods won 15, Arnold Palmer won 8, and Ben Hogan won 9. The AI model only got Gary Player right.

We will revisit the accuracy of AI models in *Chapters 7* and *9*.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/structured-output-converter-example

When we now look at how Spring AI ingests documents.

Handling documents

Spring AI provides a comprehensive and robust API for working with documents, such as PDF, DOCX, Plaintext, JSON, HTML, Markdown, along with several other document types. The core component in the API is the `Document` interface.

Understanding and ingesting different types of documents is the responsibility of Spring AI's `DocumentReader` interface. Once ingested, documents can be supplied to `DocumentTransformers` for additional processing and transformation that makes the contents of these documents easier to interpret by an AI model. Finally, the `DocumentWriter` can consume the transformed documents for storage and retrieval when necessary.

These core components can be used in a complete ETL pipeline that forms the basis for RAG.

> **Tip**
>
> A thorough treatment of Retrieval Augmented Generation, or RAG, is well beyond the scope of this book. The reader is encouraged to explore this important topic in more detail. I particularly like the explanation () from IBM researcher, Marina Danilevsky.

To put all of these concepts together, we will use Spring AI's `Document` API to build a simple ETL pipeline extracting keywords from a PDF document. Then we will summarize the contents of the PDF document overall. The PDF document we will use is an article by IBM on What is Artificial Intelligence (AI)?. I captured the contents of this article in a PDF stored in the `resources/` directory for this https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/document-etl-example.

Specifically, we want 1) a list of keywords by page number from the PDF document as a reference to where the keyword appears in the PDF and 2) to learn how the keyword is used in context to better understand its meaning.

For this example Spring AI application, we will be using Ollama and making use of Spring AI's `PagePdfDocumentReader` to ingest and *extract* pages from the PDF.

Additionally, we will use Spring AI's `KeywordMetadataEnricher` to *transform* each page of the PDF containing keyword metadata by page.

Finally, we create a custom `DocumentWriter` to output the keywords identified by the AI model to standard out (the *load* phase of ETL). Alternatively, these keywords could be stored in a database or other type of data store using Spring Data for later processing.

The ETL Pipeline can be loosely depicted as follows:

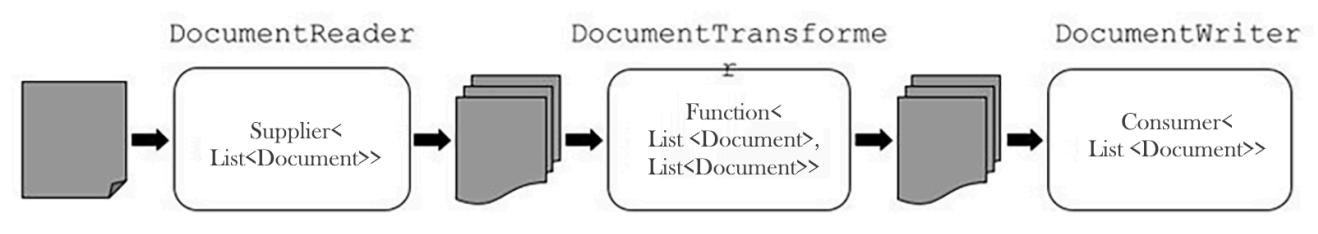


Figure 3.11: Spring AI ETL Pipeline

To start, we build our ETL pipeline with an ordered collection of Spring Boot `ApplicationRunner` beans for each phase, starting with the extract (ingest) phase:

```
AtomicReference<List<Document>> documentsReference
  = new AtomicReference<>(null);
Resource pdfDocument = new ClassPathResource(DOCUMENT_NAME);

@Bean
@Order(1)
ApplicationRunner extractRunner() {
  return args -> {
    print("Ingesting Document [%s]%n", DOCUMENT_NAME);
    PdfDocumentReaderConfig documentReaderConfiguration =
      PdfDocumentReaderConfig.builder()
        .withPagesPerDocument(1)
        .build();
    PagePdfDocumentReader pdfReader =
      new PagePdfDocumentReader(this.pdfDocument,
        documentReaderConfiguration);
    List<Document> pdfDocumentPages = pdfReader.read();
    this.documentsReference.set(pdfDocumentPages);
  };
}
```

Then, we transform the pages of the PDF document using Spring AI's `KeywordMetadataEnricher` , which enlists the help of the example application's configured AI model (Meta's Llama3.2 from Ollama) to identify the keywords in the PDF:

```
@Bean
@Order(2)
ApplicationRunner transformRunner(ChatModel chatModel) {
  return args -> this.documentsReference.updateAndGet(docs -> {
    print("Transforming Documents (%d)%n", docs.size());
    KeywordMetadataEnricher keywordMetadataEnricher =
      new KeywordMetadataEnricher(chatModel, 25);
    return keywordMetadataEnricher.transform(documents);
```

```
    });
  }
```

Finally, we output the keywords to standard out using our custom `DocumentWriter`:

```
@Bean
@Order(3)
ApplicationRunner loadRunner() {
  return args -> StdOutKeywordDocumentWriter.INSTANCE
    .accept(this.documentsReference.get());
}
```

We call on the help of our custom `DocumentWriter` to perform this task:

```
class StdOutKeywordDocumentWriter implements DocumentWriter {
  @Override
  public void accept(List<Document> documentPages) {
    print("Keywords in Document [%s]:%n", DOCUMENT_NAME);
    IntStream.range(0, documentPages.size()).forEach(index -> {
      int pageNumber = pageIndex + 1;
      Document page = documentPages.get(pageIndex);
      print("%d - %s%n".formatted(pageNumber,
        page.getMetadata().get(EXCERPT_KEYWORDS)));
    });
  }
}
```

Additionally, Spring AI provides the `FileDocumentWriter`, and you can use a `VectorStore` implementation to store the Embeddings (Vectors) generated by an AI model from your `Documents`. This further aids in RAG and *Similarity Searches* as previously discussed and demonstrated.

The output from running this program appears similar to:

```
Extracting/Ingesting Pages from Document [WhatIsArtificialIntelligenceByIBM.pdf]
... Processing PDF page: 1
... Processing PDF page: 3
... Processing PDF page: 5
... Processing PDF page: 7
... Processing PDF page: 9
... Processing PDF page: 11
... Processing PDF page: 13
... Processing PDF page: 15
... Processing PDF page: 17
... Processing PDF page: 19
... Processing 25 pages
Transforming Documents (25)
Keywords in Document [WhatIsArtificialIntelligenceByIBM.pdf] for each page:
1 - Here are 25 unique keywords for the document, formatted as comma-separated:
Artificial Intelligence, Machine Learning, Deep Learning, Generative AI, IBM AI solutions, ...
Let me know if you'd like me to help with anything else!
2 - Here are 25 unique keywords for the document, formatted as comma-separated:
Artificial intelligence, generative AI, machine learning, deep learning, artificial, intelligence, ...
Let me know if you'd like me to help with anything else!
3 - Here are 25 unique keywords in a comma-separated format:
Machine learning, AI, artificial intelligence, algorithm, prediction, decision, data, computer vision, ...
```

Full text for the AI model generated response can be found in GitHub along with the example at https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/document-etl-example.

There is no single way to chunk up your documents. Your strategy will depend on the content, your use case and AI model context window sizes. In this example we used `PagePdfDocumentReader` to split up the PDF by page. Additionally, Spring AI offers the `ParagraphPdfDocumentReader`. You can implement a custom `DocumentReader` to read and split the documents however you like. In *Chapter 7*, we will talk about Spring AI's `TextSplitter` interface.

TIP Refer to Spring AI's reference documentation for further details on Document and the ETL Pipeline.

We wrap up the chapter by reviewing Spring AI's Function calling capabilities.

Calling functions

The final stop on our Spring AI features, functionality and API journey ends with `Function` calling. This is another really neat Spring AI feature that I am partial to.

Function calling gives your Spring AI application an opportunity to be called by an AI model in response to a prompt in order to seed the AI model's generated response with additional, relevant, time-sensitive or contextually-aware information.

Again, this is another use case for **RAG** when the data is not as static as what is contained in a `Document`, but one where the information needs to be pulled from some external data source or service in real time.

The core component in Spring AI's Function Calling API is the `FunctionToolCallback` interface. This interface has been defined by Spring AI as:

```
class FunctionToolCallback implements ToolCallback {
  ToolDefinition getToolDefinition();
  ToolMetadata getToolMetadata()
  String call(String toolInput, ToolContext toolContext);
}
```

Technically, you do not need to depend on Spring AI function types. You can simply declare a plain Java `Function` to describe your callback and encapsulate the functionality required to collect the information to give back to the AI model.

We will see in a moment that the details used by the AI model to call our `Function` can be specified without depending on Spring AI types.

StockQuotesFunction application

We begin our Spring AI application as a Spring Boot application using Spring AI with Ollama and Meta's llama3.2 model. We make use of Spring AI's `ChatClient` interface to wrap the `ChatModel`, `PromptTemplate` and `Prompt` classes.

```
@Bean
ChatClient chatClient(ChatModel chatModel) {
  return ChatClient.builder(chatModel).build();
}
```

Additionally, we will be defining a Java `Function` as a managed Spring bean to make a request to fetch stock quotes using Polygon.io 's REST API. We will come back to this in a moment.

NOTE Of course, it is not strictly necessary, nor recommended, to use an AI model to request a stock quote when you can simply use a [REST] API, like the one from Polygon.io to accomplish a similar result. However, this may be useful when performing more complex analysis and attempting to forecast the price of a stock when combined with other data. If you have a pre-trained, proprietary AI model in this case, then that is not the same thing.

To make a call to Polygon.io's REST API, we make use of the core Spring Framework' s synchronous RestClient .

```
@Bean
RestClient polygonClient(Environment environment) {
  String baseUrl = environment.getRequiredProperty(
    "examples.app.stock-quotes.polygon.api.base-url");
  Consumer<HttpHeaders> defaultHttpHeaders = httpHeaders -> {
    String polygonApiKey = environment.getRequiredProperty(
      "examples.app.stock-quotes.polygon.api.key");
    httpHeaders.add(HttpHeaders.AUTHORIZATION, "Bearer %s"
      .formatted(polygonApiKey));
  };
  return RestClient.builder()
    .baseUrl(baseUrl)
```

```
      .defaultHeaders(defaultHttpHeaders)
      .build();
  }
```

To help define our `Function` so it can be invoked by the AI model and capture the JSON payload coming back in the HTTP response returned from Polygon.io's REST API call, I have defined a `StockQuote` class with internal `Request` and `Response` record types.

The `StockQuote` class is loosely defined as:

```
@Getter
class StockQuote {
  record Request(String exchangeSymbol) { }
  record Response(BigDecimal price) { }

  @Getter
  static class Result {
    @JsonProperty("o")
    private BigDecimal open;
    @JsonProperty("c")
    private BigDecimal close;
    @JsonProperty("h")
    private BigDecimal high;
    @JsonProperty("l")
    private BigDecimal low;
    //...
  }
}
```

With the `StockQuote` class definition, our Java `Function` is:

```
@Bean(STOCK_PRICE_FUNCTION_NAME)
@Description("Get the current price for a stock")
Function<StockQuote.Request, StockQuote.Response> stockQuoteFunction(Environment environment, RestClient polygonClient) {
  return request -> {
    String exchangeSymbol = request.exchangeSymbol();
    print("Fetching quote for stock [%s]...%n", exchangeSymbol);
    Map<String, ?> uriVariables = Map.of("exchangeSymbol", exchangeSymbol);
    String uri = environment.getRequiredProperty(
"examples.app.stock-quotes.polygon.api.market-data.previous-close.uri");
    StockQuote stockQuote = polygonClient.get()
      .uri(uri, uriVariables)
      .accept(MediaType.APPLICATION_JSON)
      .retrieve()
      .body(StockQuote.class);
    Assert.state(stockQuote != null, () ->
      "Failed to retrieve quote for stock [%s]"
        .formatted(request));
    BigDecimal close = stockQuote.getFirstResult().getClose();
    return new StockQuote.Response(close);
  };
}
```

`STOCK_PRICE_FUNCTION_NAME` is defined as **`StockPriceFunction`**.

Most of the logic in this `Function` involves making an HTTP request to Polygon.io's REST API to fetch a stock quote. However, we do encapsulate the stock price in a `StockQuote.Response` along with the `StockQuote.Request` forming the signature of our `Function` so the AI model knows how to call the `Function`.

Additionally, our Function bean is annotated with Spring's `@Description` annotation, which Spring AI uses to inform the AI model essentially when this `Function` should be called, such as when the user prompt is, "`What is the current price of {stock}?`"

You can see this in action in the `ApplicationRunner` bean using the `ChatClient` to prompt the AI model along with informing the model about the `Function` to call:

```
@Bean
ApplicationRunner programRunner(ChatClient chatClient) {

  return args -> {
    Scanner scanner = new Scanner(System.in);
    String stockSymbol;
    print("Enter Stock Symbol: ");
    while (isNotExit(stockSymbol = scanner.nextLine())) {
      if (StringUtils.hasText(stockSymbol)) {
        ChatOptions chatOptions = OllamaOptions.builder()
          .withFunction(STOCK_PRICE_FUNCTION_NAME)
          .build();
        Map<String, Object> promptArguments =
          Map.of("stock", stockSymbol);
        String template = "What is the current price for {stock}?";
        Prompt prompt = new PromptTemplate(template)
          .create(promptArguments, chatOptions);
        String stockPrice = chatClient.prompt(prompt)
          .call().content();
        print("AI> %s%n%n", stockPrice);
      }
      print("Enter Stock Symbol: ");
    }
  };
}
```

The program loops until the user types **exit**, allowing the user to fetch the price for multiple stocks.

The key to Function Tool Calling is the Ollama-based `ChatOptions`:

```
ChatOptions chatOptions = OllamaOptions.builder()
  .withFunction(STOCK_PRICE_FUNCTION_NAME)
  .build();
```

Our function "`StockPriceFunction`" is referred to by name using the `withFunction(:String)` build method on `OllamaOptions.Builder`.

In turn, the Ollama `ChatOptions` are then passed to the `PromptTemplate` when constructing the `Prompt`.

```
String template = "What is the current price for {stock}?";
Prompt prompt = new PromptTemplate(template)
  .create(promptArguments, chatOptions);
```

The only thing left to do is prompt the AI model and get back a stock quote or price for a given stock symbol:

```
String stockPrice = chatClient.prompt(prompt).call().content();
```

An example of the output appears as follows:

```
  .   ___          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.3)

2025-07-06T21:55:34.184-07:00  INFO 11100 --- [Stock Quotes App] [           main] p.s.a.e.f.StockQuotesFunctionApplication : Starting StockQuotesFun
Enter Stock Symbol: AAPL
Fetching quote for stock [AAPL]...
AI> The current price of AAPL is $213.55.
Enter Stock Symbol: MSFT
Fetching quote for stock [MSFT]...
```

```
AI> The current price of Microsoft (MSFT) is $498.84 per share.
Enter Stock Symbol: exit
```

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/function-example

> TIP
>
> Refer to Spring AI's reference document for more details on (Function) Tool Calling. In particular, since we were using Ollama, refer to the documentation on Function Calling with Ollama.

Before we close out *Chapter 3*, let's have a quick recap of *Chapters 2* and *3* covering common Spring AI APIs and types along with Spring AI's supported AI providers.

Spring AI API and AI provider reference

The following table summarizes APIs available in Spring AI:

API	Description
[Streaming]ChatModel	Generic interface for all AI provider models supported by Spring AI; Used to prompt the AI model and receive generated responses
ChatClient	Abstraction for ChatModel providing a higher-order interface with robust prompt handling to interact with AI provider models (recommended)
Advisor	Interface augmenting ChatClient to intercept the request sent to and response returned by an AI model; similar to Servlet Filters
ModerationModel	Interface to AI models offering moderation to detect sensitive and harmful content in text input/output
EmbeddingModel	Interface to AI models that generate embeddings (vectors), which are numerical representations of text, images, and video in order to quantify semantic meaning and derive similarity to other content
ImageModel	Interface to AI models used in image generation
TranscriptionModel	Interface to all AI models capable of transcribing audio (e.g., speech) to text
TextToSpeechModel	Interface to all AI models capable of converting text to speech

Table 3.2: Spring AI APIs

Additional Spring AI types used to support AI model interactions include, but are not limited to:

API	Description
Prompt, PromptTemplate	Classes used to construct prompts sent to the AI model; PromptTemplate can be used to conveniently parameterize the prompt with placeholder variables populated dynamically at runtime
UserMessage, AssistantMessage, SystemMessage	User and Assistant Message types represent messages from users and the AI model, respectively; SystemMessage instructs an AI model how to behave and assigns a role
Document	Interface used to model text content; often used in RAG to provide context to ground the AI model when generating a response to a user inquiry
StructuredOutputConverter	Interface used to convert generated AI model responses (e.g., JSON) into domain objects

API	Description
ToolCallback	Interface used for Tool calling (aka Function calling); Tools are functions AI models can call to acquire access to additional information; MCP is the standard protocol used to find and invoke tools, along with resources and other bits

Table 3.3: Spring AI Types

NOTE

This table is not complete. The table only represents primary APIs of Spring AI, serving as a starting point for many use cases involving AI in Spring [Boot] applications. Spring AI offers many different types to support AI interactions. Refer to Spring AI's Javadoc at https://docs.spring.io/spring-ai/docs/1.1.5/api/allpackages-index.html for a complete overview and reference to Spring AI's APIs.

The following table summarizes a few of the more common and popular AI providers supported by Spring AI:

AI Provider	Available Models	Modality	Use Case (UC)	Default Model
Amazon Bedrock	Converse, Cohere, Titan	Documents, Images, Video	Chat, Embedding, Tool Calling	UC-specific
Anthropic	Claude Haiku, Opus, and Sonnet	Text, PDF, Image	Chat, Tool Calling	claude-sonnet-4.5
DeepSeek	DeepSeek Chat	Text	Chat	deepseek-chat
Google GenAI / Vertex AI	Gemini	Text, PDF, Image, Audio, Video	Chat, Embedding, Tool Calling	UC-specific
Mistral AI	Large 3, Medium 3.5, Small 4, Codestral; See https://docs.mistral.ai/models	Text, Image, Audio	Chat, Embedding, Moderation, Tool Calling	UC-specific
Ollama	Variety of local models; see ollama.com/search	Text, Image	Chat, Embedding, Tool Calling	UC-specific
OpenAI	GPT-5.x, DALL-E, see developers.openai.com/api/docs/models	Text, Image, Audio	Chat, Embedding, Moderation, Tool Calling, Transcription, Text-to-Speech (TTS)	UC-specific

Table 3.4: Supported Spring AI Providers

NOTE

This table is not complete. To see a complete and up-to-date list of available AI providers and models supported by Spring AI, refer to Spring AI's documentation at https://docs.spring.io/spring-ai/reference/api/index.html. Also, you can refer to the Spring AI GitHub repository modules for models at https://github.com/spring-projects/spring-ai/tree/main/models. Finally, see https://docs.spring.io/spring-ai/reference/2.0/api/chat/comparison.html for a comparison of supported Spring AI Chat models.

You should now have a decent understanding of common APIs available in Spring AI and how to use them to build intelligent applications that interface with different AI providers and models supported by Spring AI.

Summary

In Chapters *2* and *3*, we covered the core features and functionality of Spring AI offered through its extensive APIs. These APIs can be used to perform a variety of tasks, such as chat, image generation, document processing, audio transcription, and much more.

Along the way we also learned a bit about different AI models, their capabilities and their limitations. Engineering effective prompts is essential in working with AI models to achieve the desired outcome. You, as the reader, are encouraged to do your own research and stay aware of the constantly evolving discipline of Prompt Engineering by using the provided resources and research available.

The APIs along with the information and examples presented in this chapter will serve as your foundation for building even more sophisticated and intelligent applications in Java using Spring AI along with the AI model of your choice.

Hopefully now you feel like you have a good understanding of Spring AI's APIs and how to use them effectively, as demonstrated in the examples. The goal is to feel confident that you could build a practical Spring AI application.

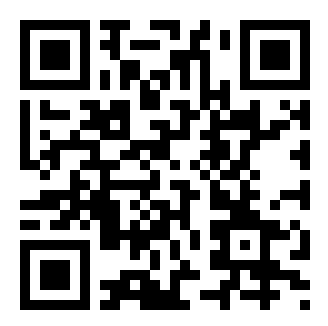
However, if you need more practice, you can build on the examples presented in this chapter. Explore and engineer different prompts. Don't just generate content, but analyze content to extract meaningful and targeted information. Leverage RAG using different data sources. Do all sorts of interesting things.

To put the material we learned to practical use, in Part 2, we will present several real-world use cases using Spring AI and different AI models to solve interesting problems. For even more in-depth knowledge of Spring AI, proceed to *Chapter 7*, "*Become a Spring AI Power User*."

In **Chapter 4**, we will learn how to build a real-time AI-powered messaging application using Spring AI, including language translation, speech-to-text, and text-to-speech capabilities, while also exploring prompt engineering, AI-generated UI development, and integration with multiple AI models and providers.

Get this book's PDF version and more

Scan the QR code (or go to https://packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.





Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

Previous chapter
‹ Chapter 2: Understanding Spring AI: Part 1

Next chapter
Part 2: Practical Spring AI ›